

Figure 7: Application's instance with four live nodes

To run the application's node, execute the *pastryClient* executable present in the current directory. *LogMsgs* is passed as an argument for the file name to which node's logs will be written to.

```
$ ./pastryClient LogMsgs
```

Figure 6 depicts the system screen after successful execution of all the above listed steps.

Initially, the *port* command is to be entered assigning an IP address and port to the node. Afterwards, enter the *create* command, which assigns a *nodeId* to the newly created node.

For example, the commands below will assign a port 9091 to the node with an IP 127.0.0.1.

```
port 127.0.0.1 9091
create
```

Similarly, you can run another node of the application on a different shell and assign a different port number. Join both the nodes using the *join* command

as follows:

```
join 127.0.0.1 9091.
```

Multiple nodes can be added to the network. *Put* and *get* queries are done using the *put* and *get* commands.

Figure 7 demonstrates four running instances of the application with port number 9091, 9092, 9093 and 9094 all running on host 127.0.0.1. After creating and joining all nodes, demonstrate the *put* (key, value) operation, run *put system1 Gaurav Arora* on Node1 where *system1* is the key and *Gaurav Arora* is the value associated with it. Then use the *hashTable* command to check which node this key-value pair gets stored in.

From Figure 7, it can be understood that the bottom-right and top-right terminal screens is where this value gets stored – i.e., Node3 and Node2, whereas the *put* operation is performed from Node1. This indicates that the messages get

stored at the node with the nearest *nodeId* compared to the message's key. Similarly, other operations like *get* have been demonstrated. Afterwards, quit one of the nodes, and see the modifications in the leaf set and neighbour set of the nodes.

Figure 8 shows logs of two different nodes. Logs corresponding to the messages populated across nodes can be seen inside the figure.

Here, eventual consistency will be achieved and from Figure 7, it is evident that the key 'system2' is replicated across three nodes while the other key has been replicated only twice. This makes the application partition tolerant and fault-resilient.

This article elaborates the design for a decentralised, self-organising, scalable and fault-resilient distributed hash table, i.e., a key-value store with no single point of failure and equal responsibility distribution between all nodes, as in the case of a P2P system.

*Continued to page 64...*